

Schemas That Do Not Fabricate

The reliability ladder, the fields that stop invention, and why valid JSON is not the same as correct data.

Fabrication is usually a **schema** problem.
The model filled a box you insisted on.

The reliability ladder

APPROACH	WHAT YOU GET
"Please return JSON"	Usually works. Fails several per cent of the time at scale.
JSON mode	Valid JSON, guaranteed. No guarantee it matches your shape.
A tool whose input_schema is your target shape	The exam's answer. Forced with tool_choice.
Grammar-constrained generation	The schema compiles to a grammar. Invalid output is unreachable.

ON THE EXAM

Each rung removes a class of failure. The top rung removes the parse problem entirely.

Why prompt-only JSON fails

- ◆ **Markdown fences** around the object, because it looks like code.
- ◆ **A friendly sentence before or after it.** Here is the JSON you asked for.
- ◆ **A field renamed or nested slightly differently** from the shape you described in prose.
- ◆ **Truncation mid-object** when the output budget runs out, which module 1 covered.
- ◆ **None of these are rare enough to ignore at volume.** One per cent of a million is ten thousand.

ON THE EXAM

Every one of these disappears at the decoding layer once generation is constrained. That is the point of the top rung.

Define a tool whose **input_schema** is your **target shape**, then force it with **tool_choice** and read the **tool_use** block.

Currency check: this is the big one

TOPIC	EXAM ANSWER	CURRENT REALITY
Schema-conformant JSON	Tool with input_schema, forced by tool_choice	Native structured outputs, generally available.
Constraining the response	Not available	output_config.format with a JSON schema constrains what Claude says.
Constraining tool arguments	Not available	strict: true on a tool constrains how Claude calls it.
Prefill	A recommended technique	No longer supported on the latest models.

ON THE EXAM

Answer the tool-schema technique on the exam. Build with structured outputs at work. Both are on this slide for a reason.

What a grammar can and cannot enforce

- ◆ **Can: types, required fields, enums, nesting, no additional properties.** The shape is guaranteed.
- ◆ **Cannot: numeric bounds, string length, recursion.** The supported schema keywords are a subset.
- ◆ **Cannot: business rules.** End date after start date, total equals the sum of the lines.
- ◆ **So you still need a validation layer.** The grammar handles shape; your code handles meaning.

ON THE EXAM

Constrained decoding moves your effort from parse repair to schema design and semantic validation. It does not remove it.

A perfectly valid record can be **entirely wrong**. Schema conformance is not accuracy.

Required, or optional?

REQUIRED

The model must produce something

- Correct when the field is always present
- Guarantees the key exists
- Absent data becomes invented data
- The commonest cause of fabrication

OPTIONAL OR NULLABLE

Absence is expressible

- Correct when the field is sometimes missing
- "Not present" becomes a valid answer
- Downstream can distinguish missing from wrong
- Costs you nothing when the data is there

THE TIEBREAKER

Mark required only what is genuinely always present. Everything else gets a way to say nothing.

Three ways to say nothing

- ◆ null: the field applies here, and the value is not in the document.
- ◆ Omitted: the field does not apply to this kind of record at all.
- ◆ An empty string is neither, and it is the worst option. It looks like data and it is not.
- ◆ Pick one convention and document it, so downstream code is not guessing.

ON THE EXAM

"Not stated" and "not applicable" are different facts. A schema that cannot distinguish them loses information.

Enums, and the escape hatch

- ◆ **An enum makes an invalid value impossible** rather than merely unlikely.
- ◆ **A closed enum with no escape forces a wrong choice** when reality does not fit your list.
- ◆ **Add an 'other' member plus a free-text detail field.** Now the model can be honest.
- ◆ **The detail field is also your roadmap.** Read the 'other' values and you learn which member to add next.

ON THE EXAM

A closed enum turns an unusual case into a miscategorised one, silently. The escape hatch turns it into a signal.

Let it tell you how sure it is

- ◆ **A confidence field per record, or per field**, alongside the value.
- ◆ **Report everything and filter downstream.** Module 12's rule, applied to extraction.
- ◆ **Add a computed flag for review**, so routing is a rule in your code rather than a judgement.
- ◆ **Confidence is not calibrated out of the box.** Tune the threshold against a labelled set.

ON THE EXAM

A low-confidence value routed to a human is useful. A silently omitted value is not.

When the document contradicts itself

- ◆ **Amended contracts, revised invoices, appended addenda.** Two values for one field, both real.
- ◆ **A single scalar field forces a choice** and throws the other value away silently.
- ◆ **Model it as a list**, each entry carrying its value and its effective date or source.
- ◆ **Let downstream decide which one applies.** That is a business rule, not an extraction decision.

ON THE EXAM

If a document can legitimately contain two answers, a scalar field is the wrong shape and no prompt will fix it.

Two fields that catch errors

- ◆ **Extract the stated total, and separately the line items.** Then compare them in code.
- ◆ **A mismatch is a signal,** and it may be a document error rather than an extraction error.
- ◆ **Ask for the source text alongside a value** where verification matters. Now a human can check in seconds.
- ◆ **Both work because they create redundancy,** and redundancy is what makes errors visible.

ON THE EXAM

This is verification in the schema. Cheaper than a review pass, and it catches a different class of error.

Normalisation belongs in the schema

- ◆ **State the format in the field description.** Dates as ISO, amounts as decimals, currency as a separate code.
- ◆ **Separate the value from its unit.** Amount and currency, quantity and measure.
- ◆ **"£1,200.00" as a string is not a number.** Someone downstream will parse it, badly.
- ◆ **Normalise at extraction, not afterwards.** The model can see the context; your parser cannot.

ON THE EXAM

The model knows that a date written 03/04 is April in one document and March in another. A regex downstream does not.

Blind retry, or informed retry?

BLIND

Send the same request again

- The model learns nothing between attempts
- Deterministic failures repeat exactly
- Costs three times as much to fail
- Module 6 warned about this shape

INFORMED

Send the validation error back

- Name the field and what was wrong with it
- The model can correct that specific value
- Converges instead of repeating
- Cap it at two or three attempts, then escalate

THE TIEBREAKER

Validate, and if it fails, tell the model exactly what failed. That is the whole loop.

Reading the stem: extraction questions

WHAT THE STEM SAYS	WHAT IT WANTS
"plausible values that do not exist"	A required field it cannot know. Make it nullable.
"forced into the wrong category"	An enum with no 'other' member
"the amended value is silently lost"	A list with effective dates, not a scalar
"valid JSON, wrong numbers"	Schema conformance is not accuracy. Add a cross-check.
"retrying produces the same failure"	Send the validation error back, not the same request
"we cannot tell missing from zero"	null versus omitted versus empty. Pick a convention.

ON THE EXAM

Almost every row points at the schema. Very few point at the prompt.

Module 13 in five lines

- ◆ The exam's answer is a tool whose input_schema is your target shape, forced with tool_choice.
- ◆ A grammar guarantees shape. It cannot guarantee meaning, so keep a validation layer.
- ◆ Required fields the model cannot know are an instruction to fabricate.
- ◆ Enums need an 'other'. Contradictory documents need a list. Values need their units.
- ◆ Retry by sending the error back, capped, then escalate.

ON THE EXAM

Seven scenarios next. Three are schema design, two are the retry loop, and two are the reliability ladder.

An invoice extraction system returns a schema with `supplier_name`, `invoice_number`, `total_amount`, `purchase_order_reference` and `due_date`, all required.

About one invoice in five has no purchase order reference. For those, the extracted value is a plausible-looking reference that appears nowhere in the document.

The prompt already says "do not guess values that are not present".

What is the most effective fix?

- ☐ A Strengthen the instruction and add an example of an invoice with no PO reference
- ☐ B Make `purchase_order_reference` nullable, and state in its description that null means not present
- ☐ C Post-process to detect references that do not appear in the source text
- ☐ D Lower the temperature so the model is less creative

◆ PAUSE AND DECIDE

What is the most effective fix?

- A

 Stronger instruction plus example
The instruction and the schema are in direct conflict. Required says produce a value; the prompt says do not. The schema wins.
- B

 Make it nullable
Removes the conflict. The model now has a way to say the reference is absent, which is the truth.
- C

 Detect it downstream
Catches the symptom after the fact, on every record, forever. The pressure to invent is still there.
- D

 Lower the temperature
Makes the invented references more consistent, not fewer. It still has to fill the field.

THE SIGNAL

"The prompt already says do not guess" is the eliminator. A prompt cannot beat a schema that demands a value.

A contract extraction schema has a `termination_notice_days` field, defined as a single integer.

Several contracts have been amended. The original states 30 days; an addendum changes it to 90. The extracted value is sometimes 30 and sometimes 90, with no indication that both exist.

Legal need to know both values and which is current.

What should change?

- ☐ A Instruct the model to always prefer the value from the most recent amendment
- ☐ B Model the field as a list of entries, each with a value, an effective date and a source reference
- ☐ C Add a second field for the amended value
- ☐ D Add a boolean `has_been_amended` flag alongside the integer

◆ PAUSE AND DECIDE

What should change?

- A

 Prefer the most recent
Discards the original, which legal explicitly need. It also makes the model arbitrate a legal question.
- B

 A list with effective dates
Both values survive, each dated, and downstream decides which applies.
- C

 A second field
Works for one amendment. A contract amended twice breaks it, and now you have two shapes of the same fact.
- D

 An amended flag
Tells you something was lost without telling you what. Better than nothing, worse than keeping it.

THE SIGNAL

"Legal need to know both" rules out every option that picks one. The field is the wrong shape, not wrongly populated.

A support ticket classifier uses an enum with six categories, defined as a closed set with no other option.

An audit finds that novel issue types are consistently assigned to whichever of the six is nearest. A new class of complaint about a recently launched feature ran for six weeks classified as "billing" before anyone noticed.

The team wants to catch new issue types earlier.

What is the most effective change?

- ☐ A Add categories for the issue types they know about now
- ☐ B Add an 'other' member with a required free-text description, and review those weekly
- ☐ C Add a confidence score and review low-confidence classifications
- ☐ D Remove the enum and allow free-text categories

◆ PAUSE AND DECIDE

What is the most effective change?

- A

 Add known categories
Fixes the issue types that already surfaced. The next novel one has the same six-week blind spot.
- B

 'other' plus description
Novel issues become visible as a category rather than hiding inside an existing one.
- C

 Confidence scores
A strong second answer, and it depends on the model being unsure. It confidently picked billing.
- D

 Free text
Removes the enum's guarantee entirely. You trade miscategorisation for unbounded category drift.

THE SIGNAL

"Consistently assigned to whichever is nearest" means it was not uncertain. That is what makes B beat C here.

An extraction pipeline validates every record against a JSON Schema. When validation fails, it resends the identical request up to three times before logging a failure.

Analysis shows that when a record fails, all three retries fail in exactly the same way. The most common failure is a date field arriving as "Q3 2026" where the schema requires a full date.

Retry costs are three times the successful-path cost on those records.

What should change?

- ☐ A Send the validation error back with the retry, naming the field and the required format
- ☐ B Increase the retry count to five
- ☐ C Loosen the schema to accept partial dates as strings
- ☐ D Add exponential backoff between the retries

◆ PAUSE AND DECIDE

What should change?

- A

 Send the error back
The model learns what was wrong and can correct that field. The loop converges instead of repeating.
- B

 More retries
Five identical failures instead of three. It multiplies the cost of the same outcome.
- C

 Loosen the schema
Makes the error go away by accepting bad data. "Q3 2026" in a date field breaks something later.
- D

 Backoff
For transient failures. This one is deterministic, as the identical repeats prove.

THE SIGNAL

"All three retries fail in exactly the same way" is the same signal as module 1 and module 6. Deterministic means structural.

A team extracts financial summaries. Every record passes schema validation. Every field has the right type.

A spot check finds that in roughly 4% of records the reported total does not match the sum of the extracted line items. Nobody had been checking, because validation was green.

They want to catch these automatically.

What is the most effective change?

- ☐ A Use grammar-constrained structured outputs so the schema is strictly enforced
- ☐ B Extract both the stated total and the line items, and compare them in code, flagging mismatches
- ☐ C Add a stricter type for the total field
- ☐ D Ask the model to double-check its arithmetic before returning

◆ PAUSE AND DECIDE

What is the most effective change?

- A

 Constrained outputs
Guarantees shape, which they already have. A grammar cannot express "total equals the sum of the lines".
- B

 Cross-check in code
A business rule needs a validation layer. The redundancy makes the error visible.
- C

 Stricter type
The type is already correct. The number is wrong, which no type expresses.
- D

 Double-check the arithmetic
Self-verification in the same context. It agrees with itself, and there is still no record of a mismatch.

THE SIGNAL

"Every record passes schema validation" is the planted detail. It rules out every answer that strengthens the schema.

A new extraction service is being built. The team's prototype asks the model to "return the result as JSON matching this structure" and pastes an example object into the prompt.

At low volume it works. In a load test of 50,000 documents, around 900 responses could not be parsed: markdown fences, a preamble sentence, and in a handful of cases a renamed field.

The team is deciding what to do before launch.

What is the most appropriate change?

- ☐ A Add a regex-based cleanup step that strips fences and preamble before parsing
- ☐ B Define the target shape as a tool input_schema and force it with tool_choice
- ☐ C Add "return only JSON, with no other text" to the prompt in capital letters
- ☐ D Retry any unparseable response up to three times

◆ PAUSE AND DECIDE

What is the most appropriate change?

- A

 Regex cleanup
Handles fences and preamble. It cannot rescue a renamed field, and it is a parser you now maintain.
- B

 Tool schema, forced
Moves the shape from prose into a structure the model is trained to respect. All three failure classes go.
- C

 Capitals
Emphasis. Module 12 covered why this is not a technique.
- D

 Retry unparseable responses
Cheaper than nothing and it treats a design problem as a transient one. The rate stays roughly the same.

THE SIGNAL

Nearly 2% at 50,000 documents is 900 failures. "It works at low volume" is how prompt-only JSON always presents.

A logistics extraction schema has a weight field defined as a string, because source documents write weights as "2.5 kg", "5 lbs" and "2500g".

Downstream, a service parses these strings to compare weights against shipping thresholds. It has been wrong three times this year, twice on pounds and once on a value written "2,500 g".

The team wants this to stop.

What should change?

- | | |
|---|---|
| <p>☐ A Improve the downstream parser to handle more formats</p> | <p>☐ B Extract a numeric weight_value and a separate weight_unit enum, normalised at extraction time</p> |
| <p>☐ C Instruct the model to always convert weights to kilograms</p> | <p>☐ D Add a validation rule rejecting weights the parser cannot read</p> |

◆ PAUSE AND DECIDE

What should change?

- A

 Better parser
Parsing downstream, where the document context is gone. Every new format is another bug found in production.
- B

 Value and unit, separated
The model reads the unit in context; your code does maths on a number. Neither has to guess.
- C

 Convert to kilograms
Better than a raw string, and it hides the original and makes the model do unit maths silently.
- D

 Reject unparseable weights
Turns silent errors into rejected records. Safer, and it still does not extract them correctly.

THE SIGNAL

Normalise where the context is. The model can see "lbs" next to the number; the parser sees only a string.